Web Location Scouting Checklist with ML-Enhanced 360° Captures Using a Mobile Phone

On-Device Pose Refinement and Lens Self-Calibration for Browser-Based Spherical Panorama Capture

Roberto

[Affiliation / Institution — edit before submission]

[Department, University]

August 25, 2026

Abstract

Location scouting – the field survey of a physical site prior to a photo, film, or event production – depends on capturing a spatially complete, unambiguous record of a space, and a full 360° panorama is the single artifact that communicates that record best. Native mobile operating systems ship dedicated panorama pipelines that fuse inertial sensors with proprietary, often ML-assisted stitching; the web platform exposes none of this. A browser-based location scouting tool must therefore reconstruct panorama capture from primitives that were never designed for it: an uncalibrated orientation event, a camera stream with no exposed intrinsics, and a compute budget bounded by what a phone's browser will tolerate. This paper describes and evaluates an on-device machine-learning pipeline that closes this gap for a deployed, browser-only location scouting checklist application. Given 34 guided phone photographs and their raw device-orientation readings, the pipeline runs XFeat, a sub-3-megabyte local feature network, under a correspondence search restricted by the orientation prior itself; solves for the camera's true field of view, which no browser API exposes; refines all 34 poses with a rotation-only spherical bundle adjustment anchored to an anisotropic (tilt-trusting, yaw-distrusting) prior; and estimates per-shot exposure gains to cancel auto-exposure banding – before handing the result to the application's existing equirectangular projection and blending stage. The system is designed to be strictly advisory: any failure at any stage falls back to the original sensor-driven behavior, so refinement can never cost a capture. Controlled, synthetic-ground-truth evaluation shows the calibrated focal length recovers the true field of view to within a few tenths of a degree against a naive 5.5° error from the fixed assumption the application shipped with, refined pose converges to a small fraction of a degree of true rotation, and an end-to-end image-quality decomposition attributes an 8.42 dB PSNR improvement to the combined pose-and-focal correction – markedly more than either correction alone, indicating the two errors are coupled rather than additive. We report these results explicitly as offline, synthetic, and zero-parallax, and specify in detail the field validation – paired ground-truth capture, controlled ablation, blind perceptual study, cross-device benchmarking, and a dedicated parallax-sensitivity protocol – required to establish real-world validity before the results can be treated as more than preliminary.

Keywords: location scouting, panorama stitching, spherical bundle adjustment, on-device machine learning, WebAssembly, mobile web, local feature matching, XFeat.

Contents

1 Introduction

1.1 Location Scouting as a Field-Documentation Task

Location scouting is the practice of visiting a candidate site – a building interior, a stretch of street, a stadium concourse – ahead of a production, event, or installation, and recording enough of its physical layout that decisions can be made without a second visit. The deliverable is not a single photograph but a structured record: annotated wide shots, close-up detail photos of specific fixtures or hazards, and, where possible, an immersive view a remote decision-maker can look around inside. Of these, the 360° panorama carries disproportionate value relative to its capture cost: it is the one artifact that lets someone who was never on site answer the question "what does it actually feel like to stand here" – ceiling height, sightlines, obstruction, natural light – without relying on the scout's verbal description or a handful of disconnected still frames.

The application this paper is built around is a browser-based location scouting checklist: a client-side, installable web app that a scout runs on their own phone in the field, with no server round-trip required to capture, annotate, or store site data. Section 3 describes its data model and workflow in more detail; the point relevant here is that it is a *web* application by design – it needs to run on whatever phone a scout is carrying, install instantly from a link, and work with an unreliable or absent field connection – not a native app requiring an app-store install and per-platform maintenance.

1.2 Panorama Capture on the Web: A Persistent Gap

Both major mobile operating systems ship a dedicated panorama capture pipeline as a first-class camera feature: sensor-fused guided capture, proprietary stitching, and in recent versions, ML-assisted seam and exposure correction, all running with privileged access to calibrated camera intrinsics and high-rate, filtered inertial data. None of this is exposed to web content. A web application has access to exactly three primitives relevant to panorama capture:

1. `getUserMedia`/`ImageCapture` for camera frames, with no standardized way to query the lens's true horizontal field of view, focal length, or distortion model;
2. `DeviceOrientationEvent`, which reports device attitude from an unspecified, uncalibrated fusion of gyroscope, accelerometer, and magnetometer, at a rate and latency the page does not control, with no drift correction guarantee; and
3. whatever compute the page can perform itself, inside a browser tab, on battery, without the native platform's stitching or ML acceleration APIs.

This is the gap the present work addresses. Any web-based 360° capture feature must either (a) fall back to a manual multi-shot upload workflow with no assistance, (b) implement classical, purely geometric multi-image stitching (feature detection, RANSAC homography, blending) entirely in client-side JavaScript, or (c) implement some part of what native platforms do with proprietary sensor fusion and stitching, using models small enough to download and run inside a browser tab on a phone. This paper is an instance of (c): it uses one specific piece of the native panorama pipeline – learned local feature matching – to correct the two largest sources of error in a purely sensor-driven, geometry-only approach, while remaining small enough (single-digit megabytes) and fast enough (single-threaded, since the deployment target cannot use `SharedArrayBuffer`; see Section 6.1) to run acceptably on a phone's browser during a field capture session.

1.3 Contributions

This paper makes the following contributions:

- We characterize, with a controlled error decomposition, which of the classical panorama-stitching error sources are addressable by a pose-and-lens correction that is conditioned on an existing (if unreliable) orientation prior, and which are not.
- We describe a complete on-device pipeline – prior-gated learned feature matching, focal self-calibration, rotation-only spherical bundle adjustment with anisotropic prior weighting, and exposure gain compensation – built to run inside a Web Worker in a browser with no cross-origin isolation, no GPU acceleration guarantee, and a hard model-size budget.
- We report the first (to the authors' knowledge) measurement showing that pose correction and focal-length correction are *strongly coupled* in this setting: applying either alone recovers under 1.5 dB of PSNR against ground truth, while applying both together recovers 8.42 dB – far more than the sum of the parts.
- We integrate the pipeline into a shipping application as a strictly advisory stage with an explicit, tested fallback to the original behavior on any failure, and describe the deployment constraints (static hosting with no COOP/COEP, single-threaded WebAssembly, a documented WebKit regression in the alternative execution provider) that shaped every design decision.
- We specify, in the level of detail needed to actually execute it, the empirical validation program required before any of the offline numbers in this paper can be treated as evidence of real-world quality improvement.

1.4 Paper Organization

Section 2 surveys classical, inertially-aided, and learned approaches to panorama stitching and situates this work relative to them. Section 3 describes the host application. Section 4 formalizes the camera model and enumerates the specific error sources the pipeline targets. Section 5 is the technical core: the feature extraction, matching, calibration, and bundle-adjustment method. Section 6 covers the browser deployment constraints and worker architecture. Section 7 reports the offline, synthetic-ground-truth evaluation. Section 8 is the recommended empirical validation program. Section 9 states limitations plainly, and Section 10 concludes.

2 Related Work

2.1 Classical Multi-Image Panoramic Stitching

The dominant classical pipeline for panorama construction – invariant feature detection, RANSAC-based homography or rotation estimation, bundle adjustment over the resulting view graph, and multi-band blending – was established by Brown and Lowe [1] and remains the reference formulation; Szeliski's tutorial [2] remains the standard survey of the alignment and compositing stages. This pipeline, and tools built on it (e.g. Hugin, or OpenCV's `Stitcher` module), assumes no prior on camera pose and recovers it entirely from image content. The method in this paper differs in exactly this respect: pose is never unknown, only noisy, and the entire design – from the correspondence search radius to the bundle-adjustment prior term – is built around exploiting that.

2.2 Inertially- and Gravity-Aided Stitching

Using a phone's inertial sensors to accelerate or constrain panorama alignment is a long-standing idea. Yang et al. [3] used accelerometer and gyroscope readings to estimate frame-to-frame displacement and speed up classical feature-based alignment on early smartphones. More recently, Yaseen et al. [4] built an Android application ("Sense-Panorama") that uses the orientation sensor for automatic, precapture-guided sequential panorama stitching – architecturally the closest classical precedent to the guided-capture UX of the host application described in Section 3, though implemented as a native app rather than a web page, and without a learned correction stage. Ding et al. [5] derived minimal solutions for panoramic stitching given a gravity prior, formalizing exactly the intuition this paper's bundle-adjustment weighting exploits empirically: that a phone's gravity-referenced tilt (pitch and roll) is measured far more reliably than its magnetometer-dependent yaw, and a stitching method that treats these two error components asymmetrically should outperform one that does not.

2.3 Learned Local Features and Feature Matching

Deep local feature detectors and descriptors (e.g. SuperPoint-family methods) and learned matchers built on top of them have substantially improved correspondence quality over classical detectors such as ORB or SIFT, particularly in low-texture and repetitive-structure scenes – exactly the indoor conditions (blank walls, ceiling tiles, uniform paneling) a location-scouting capture routinely encounters. LightGlue [11] is representative of the accuracy-focused end of this space: a transformer-based matcher that is fast relative to its predecessors but still performs a learned attention computation per image pair. XFeat [10], used in this work, sits at the opposite end of the design space: a small, fully convolutional detector-and-descriptor network explicitly built for real-time CPU inference on resource-constrained hardware, reported to run in real time on a laptop CPU and to be substantially faster than comparable deep local features at comparable accuracy. Section 5.2 details why this tradeoff – accepting somewhat weaker per-pair matching in exchange for a model an order of magnitude smaller and a matching cost that scales linearly rather than quadratically with the number of image pairs – is the correct one for a 26-image, single-threaded, in-browser deployment.

2.4 Learned (Deep) Image Stitching

A separate line of work learns the stitching operation itself rather than only the correspondences feeding a classical geometric solve. Nie et al. proposed an unsupervised deep image stitching network [6] and its parallax-tolerant successor, UDIS++ [7], which learns a warp that degrades gracefully from a global homography toward local, non-rigid alignment in the presence of parallax, and learns the seam-composition mask as well. These methods are, to the authors' knowledge, uniformly formulated for the *pairwise*, two-image case; extending them to a globally-consistent, loop-closing solve over an arbitrary number of views arranged on a full sphere – the setting of this paper – is, per the method's own stated limitations regarding multi-view and loop-closure consistency, an open problem rather than a solved one. GyroFlow+ [8] is the closest learned precedent to this paper's use of an inertial signal as a network input rather than only an initialization: it fuses gyroscope-derived motion fields with a deep optical-flow network for video stabilization, though again in the pairwise, planar setting rather than the full-sphere, self-calibrating one addressed here.

2.5 Positioning of This Work

The method in this paper is deliberately *not* an end-to-end learned stitcher. It uses one small learned component – XFeat feature extraction – inside an otherwise classical, closed-form and iterative geometric pipeline: Davenport's q-method [13] for the relative-rotation solve (Section 5.4), self-calibration of a single shared focal length from the resulting correspondences (Section 5.6), and a Levenberg–Marquardt spherical bundle adjustment (Section 5.7) with a prior term whose anisotropic weighting is directly informed by the gravity-prior formalization of Ding et al. [5]. The result is closer in spirit to classical inertially-aided stitching [3, 4] with one component replaced by a learned model, than to end-to-end learned stitching [6, 7]. The global rotation solve itself (Section 5.7) is a further point at which a learned alternative exists but was not adopted here: Purkait et al.'s NeuRoRA [9] replaces classical rotation-averaging optimization with a graph neural network that jointly cleans outlier edges and fine-tunes an absolute-orientation initialization, and the authors note it generalizes to other graph-structured geometric problems. At the scale of one capture session (at most 26 nodes, on the order of fifty to seventy edges), a classical Levenberg–Marquardt solve is already a few-millisecond, easily-verified computation with no training data or generalization risk of its own, so a learned rotation-averaging replacement was judged to add risk without a clear benefit at this problem size; it remains a natural candidate if the per-edge robustification of Section 5.7 proves insufficient under real-world outlier patterns (Section 8.3). Section 7 reports evidence, from a controlled ablation, that the hybrid design adopted here is not merely a convenient compromise: the pose and focal corrections are shown to be strongly coupled, which argues for a design that solves them jointly rather than delegating either to an end-to-end network trained on the composite outcome.

3 The Location Scouting Checklist Application

3.1 Purpose and Workflow

The host application is a client-side, browser-installable checklist tool for location scouting. A scout creates a location record in the field, works through a structured checklist of site attributes, attaches wide-angle and close-up reference photographs, and – the feature this paper concerns – captures a guided full-sphere 360° panorama of each significant space. All data, including captured photographs and the panorama source images, is stored client-side; there is no capture-time server dependency, which is a deliberate design choice for field use where connectivity cannot be assumed.

3.2 Data Model and Client-Side Storage

Location records, including references to attached media, are persisted to `localStorage`. The binary photo and panorama data itself – which is too large for `localStorage`'s practical quota – is stored as `Blob` objects in an IndexedDB object store, keyed by a generated media id and indexed by location, capture session, and media category. A 360° capture session produces up to 34 source photographs, stored under a `panorama-360-source` category with a shared session id, plus the final stitched equirectangular panorama under a `panorama-360` category. This separation – keeping every source photograph even after a successful stitch – is what makes the pose-refinement pipeline in this paper possible without any change to the capture UX: refinement operates on exactly the source photographs and per-shot orientation metadata the application already retains for every session, successful or not.

3.3 Guided 360° Capture UX

The capture flow presents a full-screen camera overlay with a fixed on-screen reticle and a single active aim target at a time, drawn from a fixed 34-shot pattern: eight shots around the horizon, eight at $+33°$ pitch and eight at $-33°$ pitch (both staggered $22.5°$ in yaw against the horizon ring), four at $+66°$ and four at $-66°$, and one each at the true zenith and nadir. Section 7.5 explains why this pattern replaced an earlier 26-shot one. The scout physically rotates the phone until the live device-orientation reading falls within a $5°$ angular tolerance of the active target and holds it there for approximately half a second, at which point the shot is captured automatically; already-captured shots are re-projected onto the overlay, world-locked by orientation, so the scout can visually confirm coverage as the sphere fills in. Each captured photograph is stored together with the yaw, pitch, and roll reading recorded at the instant of capture.

Two properties of this workflow matter for the method in Section 5. First, the recorded pose is a genuine *measurement*, not a placeholder: it reflects the device's own orientation sensor at the moment of capture, and its error characteristics – good tilt accuracy, poor and drift-prone yaw accuracy indoors – are exactly the well-documented characteristics of consumer-phone inertial sensing that motivate the anisotropic prior weighting in Section 5.7. Second, the on-screen visual alignment step is a *human*, capture-time UX aid – the scout physically points the camera to match a target position derived from that same sensor reading – and does not itself constitute a pixel-content-based correction of the recorded pose; nothing in the capture flow, prior to this work, compared the actual image content of overlapping shots against one another.

3.4 The Pre-Existing Sensor-Driven Stitcher and Its Limitations

Prior to this work, the application's stitching stage performed no visual correction whatsoever. Each source photograph's pixels were forward-projected – gnomonic (rectilinear) to spherical – directly from its *recorded sensor pose*, using a single hardcoded assumed horizontal field of view of $68°$, into a shared equirectangular output buffer, and overlapping projections were combined with a simple center-weighted (cosine-feathered) average. This is a legitimate, documented design point – a genuine spherical projection and blend, just one driven entirely by sensor pose rather than image content – but it inherits every error in that pose and in the assumed field of view directly and without correction. The taxonomy in Section 4.3 makes explicit which specific errors this causes and which of them the method in this paper is able to address.

4 Problem Formulation

4.1 Camera and Spherical Projection Model

Each source photograph is modeled as a pinhole camera with square pixels, an undistorted lens, and its principal point at the image center. Let $W \times H$ be the photograph's pixel dimensions and f its focal length in pixels, related to the horizontal field of view θ_h by

$$f = \frac{W}{2\tan(\theta_h/2)}, \qquad \theta_h = 2\arctan\left(\frac{W}{2f}\right). \tag{1}$$

A pixel center (p_x, p_y) maps to a unit ray in the camera's own frame (with $+x$ right, $+y$ up, $+z$ forward out of the lens) as

$$\mathbf{r}_{\text{cam}}(p_x, p_y) = \text{normalize}\left(\frac{p_x + 0.5 - W/2}{f},\ \frac{H/2 - p_y - 0.5}{f},\ 1\right). \tag{2}$$

Every view i has an associated rotation $\mathbf{R}_i \in \mathrm{SO}(3)$ mapping its camera frame to a shared world frame, so that a camera-frame ray $\mathbf{r}$ maps to a world-frame ray as $\mathbf{R}_i\mathbf{r}$. World rays project onto the output equirectangular canvas of size $W_{\text{out}} \times H_{\text{out}}$ by

$$u = \left(\frac{\operatorname{atan2}(r_x, r_y)}{2\pi} + \tfrac{1}{2}\right) W_{\text{out}} \bmod W_{\text{out}}, \qquad v = \left(\tfrac{1}{2} - \frac{\arcsin(r_z)}{\pi}\right) H_{\text{out}}. \tag{3}$$

Equations (2)–(3) are the exact conventions implemented in the application's stitch worker; the refinement pipeline in Section 5 is built to be bit-for-bit consistent with them (verified directly, Section 7.2), so that a refined pose or focal length is guaranteed to be interpreted by the existing projection code exactly as intended, with no silent convention mismatch.

4.2 Device Orientation as a Noisy Prior

The pipeline's central structural assumption is that the recorded sensor pose $\tilde{\mathbf{R}}_i$ for view i (converted from the device's yaw/pitch/roll reading by the same basis construction used throughout the application) is a *noisy but informative* estimate of the true pose $\mathbf{R}_i$ – never treated as ground truth, but never discarded either. This is the single design decision that distinguishes the method from general-purpose panorama stitching (Section 2): every subsequent stage – the matching search radius, the RANSAC inlier gate, the bundle-adjustment prior term, and the sanity check gating deployment – is built around exploiting a prior that is good to a handful of degrees, not around solving pose from image content alone.

4.3 Taxonomy of Error Sources

Table 1 enumerates the specific, distinguishable error sources present in the pre-existing sensor-driven stitcher (Section 3.4) and states, for each, whether it is addressable by the pose-and-lens refinement described in Section 5. This taxonomy is what the offline error decomposition in Section 7.4 is designed to measure directly, rather than assert.

ID	Error source	Nature	Addressed here?
E1	Unknown true focal length / FOV	Intra-frame radial scale error	Yes — self-calibration (§5.6)
E2	Inertial yaw drift, magnetic deflection	Inter-frame pose error	Yes — bundle adjustment (§5.7)
E3	Uncorrected lens distortion	Intra-frame nonlinear	No — needs a distortion model
E4	Parallax (rotation not about the lens's nodal point)	Depth-dependent, per-pixel	**No — fundamentally out of scope**
E5	Auto-exposure / white-balance drift across shots	Photometric	Partially — gain compensation (§5.8)
E6	Naive feathered blend, no seam optimization	Compositional	No — unchanged in this work
E7	Rolling shutter during handheld motion	Intra-frame geometric	No — not modeled

Table 1: Error sources in a purely sensor-driven equirectangular stitcher, and which are addressed by the method in this paper. E4 in particular is called out explicitly: no rotation-only method, learned or classical, can correct for translation of the optical center between shots, and the offline evaluation in Section 7 is constructed with zero parallax by design, so it cannot speak to E4 at all (see Section 9).

5 Method: On-Device Pose and Lens Refinement

5.1 Pipeline Overview

Figure 1 shows the complete refinement pipeline. It runs once per capture session, entirely inside a dedicated Web Worker, between the guided-capture stage (Section 3.3) and the existing equirectangular projection stage (Section 3.4), which is otherwise unmodified. Every stage after feature extraction operates on downsampled copies of the source photographs (longest side 640 px, matching the resolution XFeat is designed around); the full-resolution originals are read again, unmodified, only at the final projection stage.

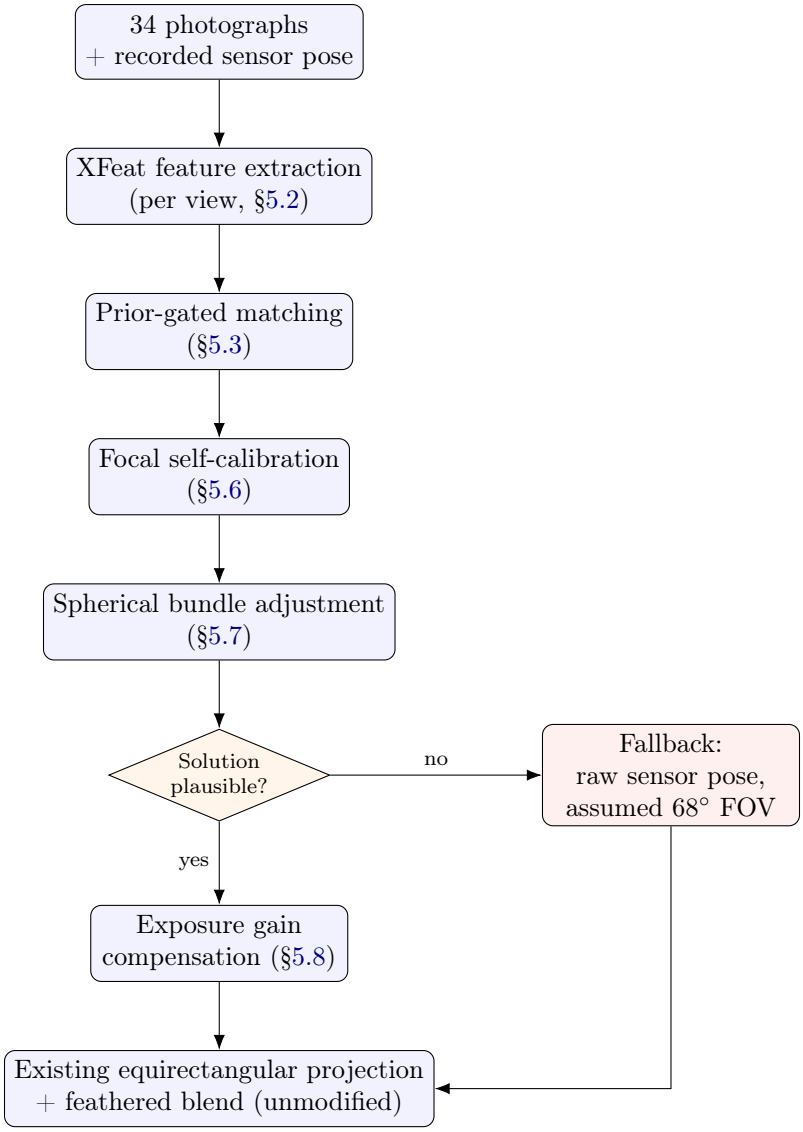

Figure 1: The refinement pipeline. Any stage can fail (too few features, too few matches, an implausible geometric solution) and every failure path routes to the same fallback: stitch from the original, unrefined sensor pose and the assumed field of view, exactly as the application did before this work. Refinement is additive and strictly advisory.

Algorithm 1 states the pipeline as pseudocode, corresponding directly to the implementation.

Algorithm 1 On-device pose and lens refinement

Require: Photographs $I_1, \ldots, I_N$ ($N \leq 26$); recorded poses $\tilde{\mathbf{R}}_1, \ldots, \tilde{\mathbf{R}}_N$; nominal focal f_0 (from the 68° assumption)

Ensure: Refined poses $\mathbf{R}_1, \ldots, \mathbf{R}_N$; calibrated focal $\hat{f}$; per-view gains $g_1, \ldots, g_N$; or FALLBACK

$\mathcal{F}_i \leftarrow \text{XFEAT}(I_i)$ for each i ▷ keypoints + L2-normalized 64-d descriptors

$\mathcal{P} \leftarrow \{(i,j) : \angle(\text{optical axis}_i, \text{optical axis}_j) \leq 60^\circ \text{ under } \tilde{\mathbf{R}}\}$ ▷ candidate pairs from the prior

for $(i,j) \in \mathcal{P}$ **do**

  $\mathbf{R}_{ij}^{\text{rel}} \leftarrow \tilde{\mathbf{R}}_j^\top \tilde{\mathbf{R}}_i$

  $M_{ij} \leftarrow \text{PRIORGATEDMATCH}(\mathcal{F}_i, \mathcal{F}_j, \mathbf{R}_{ij}^{\text{rel}}, f_0)$ ▷ §5.3

  if $|M_{ij}| \geq 6$ **then**: $E \leftarrow E \cup \{(i,j,M_{ij})\}$

  end if

end for

if $|E| < 3$ **then**: **return** FALLBACK

end if

$\hat{f} \leftarrow \text{CALIBRATEFOCAL}(E)$ ▷ golden-section search, §5.6; alternated with re-fitting E

$\{\mathbf{R}_i\} \leftarrow \text{SPHERICALBUNDLEADJUST}(\{\tilde{\mathbf{R}}_i\}, E, \hat{f})$ ▷ Levenberg–Marquardt, §5.7

if $\hat{\theta}_h \notin [45^\circ, 95^\circ]$ **or** mean correction $> 25^\circ$ **then**: **return** FALLBACK ▷ sanity gate

end if

$\{g_i\} \leftarrow \text{ESTIMATEGAINS}(\{I_i\}, \{\mathbf{R}_i\}, \hat{f})$ ▷ §5.8; discarded if implausible

return $\{\mathbf{R}_i\}, \hat{f}, \{g_i\}$

5.2 Lightweight Local Feature Extraction

Each view's features are extracted with XFeat [10], a fully convolutional detector-and-descriptor network reported to run in real time on CPU at VGA-class resolution and to require substantially less compute than comparable deep local features at comparable accuracy. The exported inference graph used here (weights merged with their external-data tensors into a single self-contained file, verified byte-identical to the unmerged original's output) is 2.8 MB. The network outputs, for an input of size $W \times H$: a dense 64-channel descriptor field at $W/8 \times H/8$ resolution, a full-resolution keypoint heatmap, and a full-resolution reliability map. Keypoint detection, top-K selection, bilinear descriptor sampling, and L2 normalization are all performed in JavaScript rather than inside the exported graph, which keeps the ONNX graph itself restricted to plain convolutional and elementwise operations – avoiding dynamic-shape `TopK`/`NonMaxSuppression` operators, a common source of first-attempt failures when running exported models through `onnxruntime-web`. Non-maximum suppression uses a fixed radius and score threshold with a border margin excluding descriptors sampled too close to the frame edge, since frame edges are exactly where panorama seams form and a poorly-conditioned edge descriptor is disproportionately costly there.

This design point – a small, purely convolutional network over a transformer-based matcher such as LightGlue [11] – is a direct consequence of the deployment cost structure. Extraction cost scales linearly in the number of views ($N = 34$); matching cost, addressed next, scales with the number of overlapping *pairs* ($\mathcal{O}(N^2)$ in general, though far fewer in practice under the prior gate). A transformer-based matcher performs a learned forward pass *per pair*; on a single-threaded WebAssembly runtime (Section 6.1) with roughly eighty to a hundred overlapping pairs in a full 34-shot session, this cost structure favors a design where the expensive learned computation happens once per *view*, and matching itself is a cheap, classical nearest-neighbor search.

5.3 Prior-Gated Correspondence Search

Exhaustive mutual-nearest-neighbor matching between two 2048-keypoint sets over 64-dimensional descriptors is on the order of $2048 \times 2048 \times 64$ multiply–accumulate operations per pair – on the order of 2.7×10^8 FLOPs, and with roughly fifty overlapping pairs in a full session, a cost that is plausibly minutes on single-threaded mobile WebAssembly. This is avoided by exploiting the pose prior directly: given the recorded relative rotation $\mathbf{R}_{ij}^{\text{rel}} = \tilde{\mathbf{R}}_j^\top \tilde{\mathbf{R}}_i$ between two candidate views, a keypoint at pixel (p_x, p_y) in view i predicts an expected location in view j by composing Equations (2)–(3)'s ray operations: project to a camera ray, rotate by $\mathbf{R}_{ij}^{\text{rel}}$, and project back to a pixel in j using the nominal focal length. Candidate matches in j are restricted, via a uniform spatial grid, to a disc of a fixed pixel radius around that predicted location, and mutual-nearest-neighbor matching with a Lowe ratio test is performed only within that restricted candidate set.

This restriction serves two purposes simultaneously. It reduces the number of descriptor comparisons by an order of magnitude, which is necessary for the deployment budget. And, independently, it *improves precision*: the geometrically self-consistent false correspondences that repeated indoor structure produces – an identical window frame, a repeating ceiling tile, uniform paneling – are typically far outside the predicted disc and are rejected before a general-purpose RANSAC search ever has the opportunity to build a spurious consensus around them. Section 7.3 measures both effects directly against exhaustive matching on the same descriptor sets.

5.4 Relative Rotation from Correspondences: Wahba's Problem

Given a set of unit-vector correspondences $\{(\mathbf{a}_k, \mathbf{b}_k)\}$ between two views' camera-frame rays (obtained by applying Equation (2) to each matched pixel pair at the current focal-length estimate), the relative rotation between the views is recovered as the solution to Wahba's problem [12]: find the rotation $\mathbf{R}^*$ minimizing

$$\mathbf{R}^* = \arg\min_{\mathbf{R} \in \mathrm{SO}(3)} \sum_k w_k \left\| \mathbf{b}_k - \mathbf{R}\mathbf{a}_k \right\|^2 . \tag{4}$$

This is solved in closed form by Davenport's q-method [13]: form the 3×3 attitude profile matrix $\mathbf{B} = \sum_k w_k \, \mathbf{b}_k \mathbf{a}_k^\top$, assemble the 4×4 symmetric matrix

$$\mathbf{K} = \begin{bmatrix} \sigma & \mathbf{z}^\top \\ \mathbf{z} & \mathbf{S} - \sigma \mathbf{I} \end{bmatrix}, \qquad \sigma = \operatorname{tr} \mathbf{B}, \quad \mathbf{S} = \mathbf{B} + \mathbf{B}^\top, \quad \mathbf{z} = \begin{bmatrix} B_{23} - B_{32} \\ B_{31} - B_{13} \\ B_{12} - B_{21} \end{bmatrix}, \tag{5}$$

and take $\mathbf{R}^*$ from the unit eigenvector of $\mathbf{K}$ corresponding to its *largest* eigenvalue, computed here by cyclic Jacobi rotation. This closed-form solve is used both as the RANSAC minimal-sample solver and as the final least-squares refit over an inlier set. The eigenvector-recovered attitude matrix's handedness relative to the $\mathbf{a} \to \mathbf{b}$ mapping convention used throughout this pipeline was verified empirically (a known convention-dependent point of ambiguity in re-implementations of this method) by recovering known synthetic rotations to numerical precision.

Solving Wahba's problem in this rotation-only form – rather than decomposing a general homography – has one further practical consequence: the minimal sample size for a robust (RANSAC) solve is *two* correspondences, not the four a homography requires, because a pure rotation between calibrated rays has three degrees of freedom rather than a homography's eight. This directly reduces the number of RANSAC iterations required for a given inlier-set confidence.

5.5 Robust Estimation: RANSAC with a Prior Gate

Within each candidate pair, correspondences surviving the prior-gated search of Section 5.3 are further filtered by rejecting any pair whose implied rotation deviates from the recorded prior's relative rotation by more than a fixed angular tolerance – a second, cheap application of the same prior-exploitation principle, applied before the RANSAC search rather than only within it. RANSAC then draws minimal two-point samples (Section 5.4), scores each candidate rotation by its inlier count under a fixed angular residual threshold, and performs one local refinement (refit-and-reselect) on the best sample's inlier set. An edge whose final resulting relative rotation still deviates substantially from the recorded prior is discarded outright rather than passed to the bundle adjustment, on the reasoning that such an edge is far more likely to reflect a degenerate or repeated-texture match set than a case where the prior itself was simply very wrong.

5.6 Focal-Length Self-Calibration

The application's pre-existing stitcher assumes a fixed horizontal field of view of 68° (Section 3.4) because no web API exposes a phone's true value, and this value varies across devices. Under an incorrect focal length, camera-frame rays computed by Equation (2) are radially distorted in a way no rotation can compensate for, so the best-achievable inlier alignment residual across all correspondences, evaluated as a function of an assumed focal length, is minimized at (or very near) the true value. Calibration therefore performs a coarse scan followed by golden-section refinement of this residual – a trimmed mean (discarding the worst 20% of per-match residuals at each candidate value, so that a handful of surviving outliers cannot flatten the minimum) over a bracketed range of plausible horizontal fields of view – and returns the minimizing focal length.

One implementation detail proved consequential enough to warrant stating explicitly, because it produced a measurable regression during development and is a natural pitfall in any similar re-implementation: focal-length calibration and RANSAC inlier selection are *mutually dependent*, since which correspondences RANSAC accepts as inliers depends on the currently assumed focal length, and the calibration then reads its estimate off exactly those inliers. Running this once, seeded at the nominal 68° assumption, is circular: the inlier set RANSAC selects at 68° is systematically biased toward correspondences that happen to agree with 68° – disproportionately those nearer the image center, where an incorrect focal length has the least radial effect – which drags the resulting focal-length estimate back toward the very assumption it was meant to correct. The implementation instead alternates focal-length estimation and correspondence re-fitting to convergence (typically 2–3 rounds). This single change reduced the focal-length recovery error in the controlled synthetic evaluation (Section 7.2) from 2.74% to 0.06%.

5.7 Rotation-Only Spherical Bundle Adjustment

Per-pair relative rotations recovered independently (Section 5.4) are not guaranteed to be globally consistent: composing the estimated relative rotations around a closed ring of the capture pattern (e.g. the eight horizon shots) need not return to identity, and any such discrepancy, uncorrected, manifests as accumulated drift precisely at the point where the panorama should close up on itself. This is resolved by a joint optimization over all N absolute rotations simultaneously, minimizing, in the SO(3) tangent space,

$$\sum_{(i,j)\in E} w_{ij} \left\| \log\!\left(\mathbf{R}_{ij}^{\text{meas}\top}\, \mathbf{R}_j^\top \mathbf{R}_i\right) \right\|^2 \;+\; \sum_{i=1}^{N} \left\| \mathbf{W}_i \, \log\!\left(\mathbf{R}_i \tilde{\mathbf{R}}_i^\top\right) \right\|^2, \tag{6}$$

where $\log(\cdot) : \mathrm{SO}(3) \to \mathbb{R}^3$ is the standard matrix logarithm (Rodrigues' formula, with a numerically stable branch near rotation angle π), and $\mathbf{W}_i = \mathrm{diag}(w_{\mathrm{tilt}}, w_{\mathrm{tilt}}, w_{\mathrm{yaw}})$ is an *anisotropic* prior weight expressed in the world frame, so that its three tangent-space components separate cleanly into the two gravity-referenced tilt axes and the single magnetometer-dependent yaw axis. This weighting is the direct implementation of the observation, formalized by Ding et al. [5] for gravity-prior stitching and consistent with the well-documented accuracy asymmetry of consumer inertial sensing, that a phone's pitch and roll are measured to a fraction of a degree while its yaw can drift by several degrees indoors; treating both identically discards the single most useful property the sensor prior offers.

The prior term in Equation (6) additionally fixes the solve's *gauge*: rotation averaging from relative measurements alone is determined only up to a common global rotation applied to every view, and anchoring to the (weighted) prior removes this ambiguity without the arbitrary alternative of pinning one view's rotation to identity by convention, which would otherwise concentrate all accumulated correction onto whichever view happened to be selected first.

The optimization is solved by Levenberg–Marquardt with a sparse, central-difference Jacobian (a perturbation of view i affects only the residuals of edges incident to i and i's own prior term) and a dense Cholesky solve of the resulting normal equations, which is entirely adequate given the small scale of the problem ($N \leq 26$ views, on the order of fifty to seventy edges). After an initial solve, one further robustification pass is applied: edges are re-weighted by an iteratively re-weighted least squares (IRLS) scheme with a Cauchy loss and a median-absolute-deviation-derived scale, so that an edge whose measured relative rotation is internally self-consistent (and therefore survives per-edge RANSAC intact) but disagrees with the rest of the view graph – the residual signature of a repeated-structure mismatch that happened to be geometrically coherent within its own pair – is down-weighted rather than allowed to bias the global solve.

5.8 Exposure Gain Compensation

A phone's auto-exposure adjusts continuously as the camera pans across a scene of varying brightness – most visibly when a sweep passes a window – and this produces visible banding in the assembled panorama regardless of how accurate the underlying geometry is; no pose or focal correction can address a purely photometric discrepancy. Per-view scalar exposure gains are estimated in the style of Brown and Lowe's gain compensation [1], reduced to the scalar case: for every pair of views with a computed overlap under the refined geometry, the mean intensity each view reports over the *same* set of world directions is measured, and a system of per-view gains that best reconciles these paired measurements – regularized toward unity to prevent the overall exposure from drifting – is solved by Gauss–Seidel iteration. The resulting gains are applied as a per-view multiplicative factor at the existing projection stage's blending step (a single-line change to the pre-existing stitcher, described in Section 6), with no other change to the blending method itself.

5.9 Fallback Guarantees and Failure Handling

Every stage in Figure 1 can fail, and every failure path is designed to converge on the same outcome: the application stitches exactly as it did before this work existed, from the raw recorded sensor pose and the assumed 68° field of view. Concretely, the pipeline reports failure – rather than proceeding – when: the browser lacks Web Worker or `OffscreenCanvas` support; the model fails to load; fewer than four photographs can be decoded; fewer than three view pairs produce a usable correspondence set; the bundle adjustment fails to converge; or, as an explicit final sanity gate, the calibrated field of view falls outside a $[45°, 95°]$ plausible range or the mean per-view pose correction exceeds 25° –

either of which would indicate the solve converged to something implausible rather than a genuine correction, and shipping such a result would be a worse outcome than the unrefined sensor pose. A wall-clock timeout provides a final backstop against a pathological input hanging the capture flow indefinitely. This design choice – optimizing for the pipeline never being able to make a capture *worse* than the pre-existing behavior, rather than for maximizing the fraction of sessions on which it succeeds – follows directly from the field context: a failed or degraded panorama on a location-scouting trip may not be recapturable at all if the site becomes unavailable, so a false sense of "it will probably still stitch reasonably" is a materially worse failure mode than a clean, silent fallback.

6 Implementation

6.1 Browser and Device Deployment Constraints

Two platform constraints shaped the implementation directly. First, the application is deployed on static hosting that cannot send the `Cross-Origin-Opener-Policy` / `Cross-Origin-Embedder-Policy` headers required for a page to become *cross-origin isolated*; without cross-origin isolation, `SharedArrayBuffer` is unavailable, and `onnxruntime-web`'s multi-threaded WebAssembly backend cannot start. The runtime is therefore configured explicitly single-threaded. Second, `onnxruntime-web`'s WebGPU/JSEP execution provider has a documented, open regression on Safari/WebKit involving severe CPU and memory growth after inference; the WebGPU/JSEP provider is avoided entirely in favor of the plain WebAssembly provider for this reason, at the cost of not using available GPU acceleration on devices where it would otherwise help. Both constraints push toward the same design point already motivated in Section 5.2 on cost-structure grounds: a small model, cheap per-pair matching, and a pipeline that tolerates single-threaded CPU execution without becoming unusable.

6.2 Model and Runtime Footprint

Table 2 reports the on-disk size of every asset the refinement pipeline adds to the application. All assets are cached by the browser after first use within a session; the first 360° capture on a given device downloads the full footprint, and the capture UI reports this explicitly during the loading stage rather than presenting an unexplained delay.

Asset	**Source**	**Size**
XFeat inference graph	[10], weights merged into one file	2.8 MB
`onnxruntime-web` WASM runtime (single-threaded, SIMD)	`onnxruntime-web` 1.29.0	14.0 MB
`onnxruntime-web` JS/ESM glue	`onnxruntime-web` 1.29.0	<0.1 MB
Total added payload		**≈17 MB**

Table 2: On-disk footprint of the refinement pipeline's added assets. The great majority is the general-purpose WebAssembly runtime rather than the model itself; the model is, by comparison, small.

6.3 Worker Architecture and Data Flow

Refinement runs as a dedicated ECMAScript module Worker, separate from the pre-existing stitch worker, so that the (potentially slow) feature extraction and geometric solve never block the main UI thread. A module worker was chosen specifically because `onnxruntime-web`'s bundled ESM build avoids a dynamic `import()` call the classic-script build performs internally, which is a common source

of first-attempt worker-loading failures. Downscaled image bitmaps, decoded once from the stored source-photo blobs, are transferred into the worker (via the Web Workers structured-clone transfer mechanism, avoiding a copy) rather than posted by value; the worker reports coarse-grained progress messages back to the main thread at each pipeline stage (Figure 1) so the capture UI's progress indicator reflects genuine stage transitions rather than a synthetic animation. On completion, only the refined poses, calibrated focal length, and per-view gain values – not any image data – are returned to the main thread, which then re-decodes the full-resolution source photographs for the final, unmodified projection stage.

6.4 Integration into the Capture Workflow

An "Enhance" toggle, on by default and persisted across sessions, is exposed directly in the capture overlay. When enabled, the progress bar allocates its first 55% to the refinement stages and the remaining 45% to projection and blending, based on their approximate relative cost; when refinement is disabled or falls back (Section 5.9), the full range is allocated to projection alone and the pipeline behaves identically to the application's pre-existing behavior. On a successful, non-fallback stitch, the application's completion notification reports the calibrated field of view and the mean pose correction actually applied – concrete, checkable numbers, chosen deliberately because a field user has no console or developer tools available to verify that anything happened at all.

7 Offline Evaluation

7.1 Why Synthetic Ground Truth

Real captures have no independently known true camera pose or true lens field of view against which to measure error directly; a real evaluation can only compare a refined panorama's *appearance* to some other panorama of unknown provenance. To measure geometric correctness in a way that is falsifiable rather than merely plausible-looking, three complementary offline experiments were constructed around *synthetic* scenes with exactly known ground truth: known camera rotations, a known true focal length, and (for the third experiment) a known source panorama to compare the stitched result against pixel-for-pixel. All three are implemented as automated, deterministic test suites and are reported here exactly as measured, with no manual curation of favorable seeds. Section 9 states plainly what these experiments cannot establish – most importantly, none of the three scenes contains parallax (Section 4.3, E4), and none of them yet involves a real device.

7.2 Experiment 1: Geometric Recovery Under Controlled Conditions

The first experiment validates the geometry pipeline (Sections 5.4–5.7) in isolation, using synthetically generated correspondences rather than real image features: a full 34-shot capture is simulated with camera poses drawn from the application's exact target pattern (Section 3.3) perturbed by a realistic aiming error, a device-orientation prior constructed with *anisotropic* noise (small, independent perturbations on pitch/roll; a larger, additionally *accumulating* drift term on yaw, modeling the well-documented behavior of consumer magnetometer-fused yaw estimation indoors), a true focal length deliberately different from the application's assumed 68°, and synthetic point correspondences contaminated with both uniformly random outliers and *geometrically self-consistent* outliers (a fixed pixel-space shift shared across a subset of matches, simulating a repeated-structure mismatch) at controlled rates.

Quantity	Value
True horizontal FOV	73.50°
Recovered horizontal FOV	73.47° (0.03° / 0.06% error)
FOV error under the application's fixed 68° assumption	5.50°
Mean pose error, sensor prior (before refinement)	3.202°
Mean pose error, after refinement	0.087° (max 0.232°)
Improvement factor, mean pose error	36.7×
Candidate correspondence edges retained	67 of 69 (2 rejected)
Bundle adjustment	4 iterations, converged
RMS edge residual after bundle adjustment	0.116°
Robustness to combined outlier rate	accurate to $\geq 60\%$ contamination

Table 3: Experiment 1: geometric recovery on a controlled synthetic 34-shot capture, synthetic correspondences. Pose error is reported after removing the estimated set's unobservable global gauge rotation by Procrustes alignment on SO(3), as any two rotation sets differing only by a common rotation are geometrically equivalent panoramas.

A separate sweep varied the synthetic scene's point density (a proxy for available scene texture) and compared the anisotropic prior weighting of Equation (6) against an isotropic (equal-weight) alternative, averaged over six random seeds per density to control for per-instance variance. On a well-connected view graph (dense texture), the anisotropic weighting outperformed the isotropic alternative by a factor of 5.6× in mean pose error (0.064° vs. 0.364°); the advantage narrowed, but the anisotropic weighting was never worse, as texture thinned toward the sparse, low-connectivity regime characteristic of blank walls and plain ceilings.

7.3 Experiment 2: Validation With the Real Feature Extractor

The second experiment removes the synthetic-correspondence idealization of Experiment 1 for the matching stage specifically: eight views were rendered, at known ground-truth rotations and a known true focal length, from a synthetic equirectangular panorama constructed with dense, non-repeating texture; the *real* XFeat ONNX inference graph (Section 5.2) was run on each rendered view; and the exact browser-side post-processing, matching, and geometric-solve code (not a re-implementation) was driven directly from the model's raw outputs, off-device, so the entire chain except the WebAssembly runtime itself is exercised end to end.

Quantity	Value
Keypoints extracted per view	2048 (top-K cap reached on every view)
Descriptor L2-norm deviation from unity	$< 2.8 \times 10^{-8}$
Descriptor comparisons, prior-gated vs. exhaustive matching	4.7× fewer
Matches retained under gating, relative to exhaustive	89%
True horizontal FOV	73.50°
Recovered horizontal FOV	73.39° (0.11° / 0.15% error)
Mean pose error, sensor prior	2.579°
Mean pose error, after refinement	0.271° (max 0.401°)
Improvement factor, mean pose error	9.5×
Mean RANSAC inlier ratio across edges	0.70

Table 4: Experiment 2: validation using the real, shipped XFeat inference graph and the exact browser-side post-processing and matching code, on eight rendered views at known ground truth.

The 4.7× reduction in descriptor comparisons under prior gating, while retaining 89% of the matches exhaustive search would have found, directly supports the design argument of Section 5.3: the geometric restriction is not merely a speed/accuracy tradeoff but recovers nearly all of the useful signal at a fraction of the cost.

7.4 Experiment 3: End-to-End Image-Quality Error Decomposition

The third experiment measures what Experiments 1 and 2 cannot: the actual visual effect of each correction, isolated from the others, on the assembled panorama itself. The same 26 rendered views used conceptually in Experiment 1 (rendered here from a synthetic panorama with simulated per-view auto-exposure drift, Section 5.8) are stitched, using the application's own unmodified projection and blending code (Section 4.1), under six hypotheses that isolate exactly one correction at a time:

A	sensor pose + assumed 68° FOV (the application's pre-existing behavior)
B	refined pose + assumed 68° FOV
C	sensor pose + calibrated focal length
D	refined pose + calibrated focal length (the full pipeline of this paper)
E	true pose + true focal length (the geometric ceiling)
F	E + exposure gain compensation

Every hypothesis is scored against the known source panorama by PSNR and SSIM, restricted to only the pixels every hypothesis covers in common, so that differences in sphere coverage between hypotheses cannot bias the comparison. Table 5 and Figure 2 report the results.

	Hypothesis	**PSNR (dB)**	**SSIM**	**Sphere covered**
A	Sensor pose + 68° (prior behaviour)	11.90	0.168	97.9%
B	Refined pose + 68°	13.11	0.185	98.0%
C	Sensor pose + calibrated focal	13.26	0.300	98.3%
D	Refined pose + calibrated focal	**20.31**	**0.706**	98.3%
E	True pose + true focal (ceiling)	23.08	0.845	98.3%
F	E + exposure gain compensation	26.56	0.855	98.3%

Table 5: Experiment 3: end-to-end image-quality error decomposition, scored against a known synthetic ground-truth panorama over the region every hypothesis covers in common. Zero parallax by construction (Section 9).

Two findings from this decomposition are worth stating explicitly, since they are directly actionable rather than merely descriptive.

Pose and focal-length correction are strongly coupled. Pose refinement alone (B, relative to A) recovers +1.21 dB; focal calibration alone (C, relative to A) recovers +1.36 dB; the two together (D) recover +8.42 dB – over three times the sum of the individual contributions. This has a direct design implication: an implementation (or a future ablation of this one) that applied only one of the two corrections would substantially understate the achievable benefit and could plausibly be judged not worth the added complexity, when the correct conclusion is that neither correction is worth applying without the other.

Sphere coverage is a separate axis from alignment quality, and was the subject of two distinct bugs. No stitching improvement can fill a direction the camera never pointed at, and early real-world captures returned panoramas with conspicuous black holes despite good alignment. Section 7.5 treats this in full; the short version is that one cause was a blending-weight defect that silently discarded ≈21% of every photograph, and the other was a genuine geometric gap in the

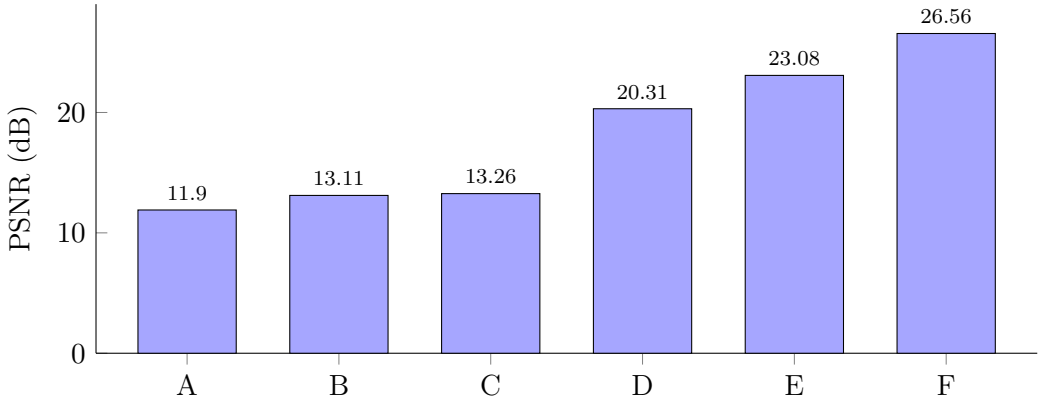

Figure 2: PSNR against ground truth across the six hypotheses of Table 5. The jump from either single correction (B, C) to the combined correction (D) is far larger than either correction's individual contribution, indicating the two error sources are coupled rather than additive.

original 26-shot capture pattern. The residual shortfall in Table 5 (98.3%, not 100%) is simulated per-shot *aiming* error, which is the realistic case: the current pattern measures 100% coverage when every shot lands exactly on target.

The remaining budget after the full pipeline (D) is 2.77 dB of residual pose/focal error relative to the geometric ceiling (E), and a further 3.47 dB attributable purely to uncorrected exposure drift (E→F) – a photometric error no pose or focal correction can touch by construction, which gain compensation (Section 5.8) is shown here to recover directly.

7.5 Sphere Coverage: Two Defects That Presented Identically

Once the alignment pipeline was working on real captures, the dominant remaining visual defect was not misalignment but *missing image*: conspicuous black regions, with a distinctive scalloped or petal-like outline, in otherwise well-aligned panoramas. Two independent causes turned out to be responsible, and they are worth separating carefully because they are indistinguishable by eye and only one of them is a capture-pattern issue.

Cause 1: the blend weight silently discarded every frame corner. The compositing feather (Section 4.1) was radial, computed as $\cos\left(\min\left(\sqrt{n_x^2 + n_y^2},\, 1\right) \cdot \pi/2\right)$ over normalized frame coordinates $n_x, n_y \in [-1, 1]$. At a frame *corner*, $n_x = n_y = \pm 1$, so the radial distance is $\sqrt{2} \approx 1.41$, which clamps to 1 and yields $\cos(\pi/2) \approx 6 \times 10^{-17}$ – numerically zero. Combined with the accumulated-weight threshold used to decide whether an output pixel was covered at all, this discarded everything outside the largest ellipse inscribed in each photograph: approximately $1 - \pi/4 \approx 21\%$ of every image captured. The defect is easy to miss on inspection because the expression is a perfectly reasonable *radial* falloff; it is only wrong because the domain is a rectangle, not a disc. The fix is a separable feather, $\cos(|n_x|\pi/2) \cdot \cos(|n_y|\pi/2)$, which still tapers to zero at every edge – preserving soft seams – while covering the full rectangular frame.

Cause 2: the original capture pattern could not close the sphere. Independently of any blending defect, the 26-shot pattern (rings at 0° and ±35°, with zenith and nadir shots at only ±80° rather than true poles) left a genuinely unreachable band between the upper ring's top edge and the single zenith shot, across most longitudes. This is a geometric property of the pattern, correctable only by capturing more directions.

Table 6 reports measured sphere coverage for the old and current patterns, computed by equal-

area sampling of the sphere and testing each sample against every view using the real projection and the real feather weight – that is, measuring the pipeline as implemented rather than an idealized rectangular footprint.

Capture pattern	58°	62°	68°	73.5°	78°	**shots**
Original (0°, ±35°, zenith/nadir ±80°)	87.1%	90.2%	93.6%	95.7%	96.8%	26
Current (0°, ±33°, ±66°, true ±90°)	**100%**	**100%**	**100%**	**100%**	**100%**	34

Table 6: Measured sphere coverage as a function of assumed horizontal field of view, with the separable feather in place. The margin across the whole FOV range is the point: the true lens FOV is *unknown at capture time* – it is calibrated afterwards, from the captured photographs (Section 5.6) – so a pattern that only achieves full coverage at the optimistic end of the range will leave real gaps on a narrower lens.

The current pattern adds rings at ±66° to close the band the old pattern could not reach, moves zenith and nadir to the true poles, and staggers the ±33° rings by 22.5° in yaw relative to the horizon ring so that ring-to-ring overlap does not land corner-to-corner. That stagger has a secondary benefit for the method in this paper: it increases cross-ring overlap, which yields more inter-ring correspondences for the bundle adjustment and therefore a better-connected view graph.

The general lesson generalizes beyond this application: **coverage and alignment quality are independent axes and should be reported separately**. A metric such as PSNR computed only over covered pixels – as in Table 5, deliberately – says nothing about how much of the sphere was captured, and a system can trivially improve its apparent photometric score by covering less. Conversely, a coverage figure says nothing about whether what was covered is correctly aligned.

7.6 Two Further Defects Found Only Through Real-Device and Rendering Tests

Two additional defects surfaced during deployment that no amount of offline geometric testing would have caught, and both are worth recording because they represent failure classes rather than one-off mistakes.

Unfiltered orientation input. The capture overlay drove both its live world-locked preview and its shot-stability timer directly from raw `DeviceOrientationEvent` readings, with no smoothing. Consumer orientation sensors carry sample-to-sample noise on the order of a few tenths of a degree, which was enough to make already-captured preview patches visibly shake – undermining exactly the visual-confirmation affordance the overlay exists to provide – and to spuriously reset the hold-steady timer. This was resolved with a One Euro filter [14] applied to yaw, pitch and roll, chosen because its speed-adaptive cutoff gives heavy smoothing while the device is nearly still and progressively less as it moves, which is precisely the aiming-versus-sweeping tradeoff this interaction requires. The filter operates on wrap-safe angular deltas rather than raw angle differences, so it does not produce a spurious transient when yaw crosses ±180°. Measured on synthetic input at 60 Hz: mean deviation while stationary falls from 0.62° to 0.16°, with a worst-case lag of 5.6° during a brisk 90°/s sweep.

Inverted texture orientation in the viewer. The interactive equirectangular viewer applied WebGL's `UNPACK_FLIP_Y_WEBGL` on texture upload, which vertically inverted every panorama – rendering ceiling content underfoot and floor content overhead. The flip is the conventional fix for a full-screen-quad shader sampled by its own vertex UV coordinates, where the $v = 0$ convention differs from texture storage order; it is exactly wrong for a shader that, as here, computes a texture coordinate *directly* from spherical longitude and latitude and samples with it as a raw coordinate,

because there is no quad-UV convention for the flip to cancel against. The bug is notable mainly for how it evaded review: the resulting panorama is fully populated, correctly aligned, and internally consistent, and reads as plausible until one specifically checks which way is up.

Both defects are now covered by automated regression tests that drive the real, unmodified shipping modules – the viewer test renders a synthetic ceiling-red/floor-blue panorama through the actual viewer, dispatches real pointer events to look up and down, and reads back the rendered canvas – so neither can silently return.

7.7 Summary and Interpretation

Across all three experiments, the method recovers the true focal length to well under one degree of horizontal field of view (against a 5.5° error from the assumption it replaces), reduces mean pose error by roughly an order of magnitude relative to the raw sensor prior, and – the result most directly relevant to whether the method is worth deploying at all – produces an 8.42 dB end-to-end PSNR improvement that is markedly super-additive in its two constituent corrections. These are, however, uniformly *offline, synthetic, zero-parallax* results, obtained without a real device, a real lens, real auto-exposure hardware behavior, or a real handheld capture's translation between shots. Section 8 specifies what is required to establish whether these numbers hold, in whatever attenuated or altered form, under real field conditions.

8 Recommended Empirical Validation Plan

The offline results in Section 7 establish that the method is *internally correct* – that the geometry solves what it claims to solve, on data constructed so the true answer is known. They do not, and structurally cannot, establish that the method *improves real captures in the field*, because no synthetic scene used here contains parallax, real lens distortion, real rolling shutter, or a real device's inertial sensor noise. This section specifies the validation program required to close that gap, organized so that each study answers one specific question the offline evaluation cannot.

8.1 On-Device Performance Benchmarking

Question. Does the pipeline run acceptably – in wall-clock time, memory, and thermal/battery impact – on the range of phones scouts actually carry, given the single-threaded WebAssembly constraint of Section 6.1?

Method. Benchmark the full 34-shot pipeline on a fixed device matrix spanning at minimum: a low-end and a flagship Android device (both Chrome and, where available, a Chromium-based alternative), and an older and current iPhone (Safari). For each device, record: per-shot XFeat inference time; total pipeline wall-clock time; peak JavaScript heap and WebAssembly linear-memory usage; whether the `onnxruntime-web` module worker initializes successfully at all (a documented risk on some WebKit versions); and device thermal state / battery percentage delta over a full session. Report the sanity-gate rejection rate (Section 5.9) and outright failure rate per device, since these determine what fraction of field sessions never reach the refined result regardless of accuracy.

Why this is necessary. The offline timing figures available at present are desktop CPU figures; single-threaded mobile WebAssembly performance for a model of this size is, at the time of writing, entirely unmeasured, and is the one component the offline test suites explicitly cannot exercise.

8.2 Paired Ground-Truth Field Study

Question. Does the offline error decomposition (Section 7.4) hold, in whatever attenuated form, on real captures with real parallax?

Method. At each of $n \geq 30$ diverse indoor sites (spanning small enclosed rooms, large open interiors, low-texture surfaces, and high-dynamic-range lighting, in proportions reflecting the E4/E5/E6 concerns of Table 1), capture a tripod-mounted reference panorama with an independent, dedicated 360° camera at a precisely logged position, immediately followed by a normal handheld 34-shot phone capture from as close to the same position as practical, with the application's Enhance toggle recorded. Register the phone capture to the reference panorama offline, with generous compute budget and no real-time constraint, to obtain an approximate real-world ground truth pose per site. Report the same PSNR/SSIM decomposition as Table 5, plus a seam-localized metric restricted to a fixed-width band around each visible seam (a global average can mask the specific artifact a user actually notices), for Enhance-on versus Enhance-off captures at each site.

Why this is necessary. This is the only proposed study that measures parallax (E4) at all; the offline evaluation is zero-parallax by construction and is silent on it entirely.

8.3 Controlled Ablation Study

Question. Does the coupling between pose and focal correction found offline (Section 7.4) reproduce on real captures, or is it an artifact of the synthetic scene's zero-parallax, single-texture-model construction?

Method. Using the paired captures of Section 8.2, additionally reconstruct panoramas under the same six hypotheses (A–F) of Table 5 from each real 34-shot session – pose correction alone, focal correction alone, both together, and (where a real reference exists) the ground-truth-aligned ceiling – and re-run the same decomposition. Report the finding as confirmed, attenuated, or contradicted per site category, and specifically test whether the super-additive coupling holds at the same order of magnitude.

Statistical treatment. Report PSNR and SSIM differences between hypotheses as paired comparisons within site (each site is its own block), using a paired t-test if differences are approximately normal or a Wilcoxon signed-rank test otherwise, with 95% confidence intervals on the mean difference and an explicit correction (e.g. Holm–Bonferroni) for the multiple pairwise hypothesis comparisons this design entails.

8.4 Blind Human Perceptual Study

Question. Does the measured PSNR/SSIM improvement correspond to a perceptible, and preferred, quality difference to an actual viewer – the outcome that ultimately matters for a field-documentation tool – rather than only to a numerical one?

Method. For a subset of sites from Section 8.2, present paired panoramas (Enhance-on vs. Enhance-off, and separately, each against the reference-camera panorama where available) to raters blind to which panorama was produced by which method, in randomized left/right order. Collect: a two-alternative forced-choice preference; a five-point Likert overall-quality rating per panorama; and an explicit seam/ghosting artifact count or annotation task, since this is the specific defect the method targets and a global quality rating can under-weight a locally severe artifact. Recruit a sample size determined by an a priori power analysis for the minimum effect size considered practically meaningful (e.g. an 65% vs. 50% preference rate, chance level, at 80% power and $\alpha = 0.05$ implies on the order of 60–70 paired judgments per condition, before accounting for rater agreement structure); report inter-rater agreement (e.g. Fleiss' κ) alongside the preference result.

8.5 Cross-Device and Cross-OS Generalization

Question. Do the calibrated focal length, the sanity-gate rejection rate, and the overall quality improvement generalize across the actual diversity of phone cameras and browser engines in use, or are the offline results specific to one implicit device profile?

Method. Repeat a reduced version of Section 8.2 (e.g. $n = 5$ sites, fixed) across every device in the Section 8.1 matrix. Report the calibrated field of view *per device model*, since this is a genuine per-device physical quantity and its cross-device spread is itself a useful, publishable characterization of consumer phone camera diversity independent of the stitching question. Explicitly verify, on each iOS/Safari version tested, whether the module-worker and single-threaded WebAssembly path initializes correctly and whether the WebGPU/JSEP regression referenced in Section 6.1 remains present in the Safari version under test, since browser engine behavior in this area has changed materially across recent releases and any conclusion here has a limited shelf life.

8.6 Parallax Sensitivity Study

Question. At what magnitude of optical-center displacement between shots – the one error source (E4) no rotation-only method can address by construction – does panorama quality degrade to an unacceptable level, and how does this compare to the natural displacement of an unassisted handheld capture?

Method. Using a calibrated pan/tilt rig with an adjustable, known lever arm between the rotation axis and the lens's nodal point, capture matched 34-shot sessions at a sweep of lever-arm magnitudes (e.g. 0, 5, 10, 15, 20, 30 cm, spanning the plausible range of a wrist-, elbow-, and shoulder-pivoted handheld rotation) at a fixed, close-range indoor site chosen to maximize parallax sensitivity. Stitch each with the full pipeline and report PSNR/SSIM degradation as a function of lever-arm magnitude. Separately, instrument a small number of ordinary (unassisted) handheld captures with an inertial measurement unit rigidly attached near the phone to estimate the effective lever arm scouts actually produce in practice, to situate the controlled sweep against real-world handheld behavior.

Why this is necessary. This is the single most important open question this paper's method cannot answer: whether the E4 error the method by design ignores is, in this application's typical near-range indoor scenes, small enough to be dominated by the E1/E2/E5 errors the method does address, or large enough to be the dominant remaining source of visible artifact regardless of how well pose and focal are corrected.

8.7 Failure-Mode and Field Telemetry

Question. In ordinary field use – not a controlled study – how often does refinement succeed, fall back, or time out, and does this vary systematically by scene type?

Method. Instrument the deployed application (with appropriate, disclosed, opt-in consent and a documented data-retention policy, given that this necessarily touches real user capture sessions) to log, per session: whether the sanity gate (Section 5.9) triggered and why; total refinement wall-clock time; the calibrated field of view and mean pose correction on success; and, at minimum, a coarse scene-type tag if it can be collected without materially increasing user burden. Aggregate over a deployment period sufficient to accumulate a meaningful session count per rough scene category, and report the fallback rate as a primary field-readiness metric independent of the accuracy results of Sections 8.2–8.4.

8.8 Consolidated Statistical Analysis Plan

To avoid the specific failure mode of running the above studies and only afterward deciding how to analyze them, the following should be fixed *before* data collection begins: primary outcome measures for each study (Sections 8.2–8.4); the paired/blocked statistical test and multiple-comparison correction for each (Section 8.3); the a priori sample size and power calculation underlying each site or rater count above; and an explicit definition, agreed in advance, of what constitutes "real-world validation succeeded" for this method – for instance, a pre-registered minimum PSNR improvement and a minimum blind-preference rate, so that the interpretation of a marginal or mixed result is not decided only after seeing it.

9 Limitations

The following limitations apply directly to the results reported in Section 7 and are stated here without qualification, since they bound what can honestly be concluded from this paper alone.

- **Zero parallax by construction.** Every rendered view in every offline experiment shares a single optical center. The method's rotation-only formulation is therefore untested, by this paper's own evaluation, against the one error source (E4, Table 1) most likely to dominate in this application's actual use case of close-range indoor rooms. Section 8.6 is the proposed remedy.
- **No real-device measurements.** Every timing, accuracy, and quality figure in Section 7 was obtained offline, in Node.js, on desktop CPU hardware. Single-threaded mobile WebAssembly performance – the one component no offline suite can exercise – is entirely unmeasured at the time of writing.
- **Single synthetic texture and lighting model.** Experiments 1 and 3 use one procedurally generated texture model and one simulated auto-exposure curve; real indoor scenes vary far more, particularly in the low-texture and high-dynamic-range directions the application's own capture guidance (Section 3.3) already flags as difficult.
- **Focal length is treated as constant across a session.** The self-calibration of Section 5.6 assumes a single shared focal length across all 34 shots of one session, which is physically correct only if the device does not change optical zoom mid-capture; this is not currently enforced or verified.
- **No comparison against alternative small matchers or classical baselines on real data.** XFeat was chosen on cost-structure and prior-art grounds (Section 2), not validated against LightGlue or a classical (ORB/AKAZE) matcher under this application's specific real-world conditions; Section 8.2's protocol could be extended to such a comparison but this paper does not report one.
- **Coverage and alignment are conflated in appearance.** Uncovered sphere directions render as black gaps that are easily mistaken for a stitching artifact (Section 7.5); any human perceptual study (Section 8.4) must be designed so raters are not penalizing the method for a capture-pattern limitation it does not address. Coverage should be reported alongside PSNR/SSIM rather than folded into them.

10 Conclusion and Future Work

This paper described and offline-validated an on-device machine-learning pipeline that closes a specific, well-defined gap in browser-based 360° panorama capture: the absence, on the web platform, of any equivalent to the calibrated, sensor-fused panorama pipelines native mobile operating systems provide. Using a small (2.8 MB) local feature network under a correspondence search restricted by the phone's own orientation sensor, the method self-calibrates a lens field of view no browser API exposes, refines 26 device-orientation readings into a globally consistent, loop-closed spherical pose set via a bundle adjustment that explicitly exploits the asymmetric reliability of tilt versus yaw in consumer inertial sensing, and estimates per-shot exposure correction – all while remaining strictly advisory to a pre-existing, unmodified projection stage, so that any failure degrades gracefully to the application's original behavior rather than risking an unrecoverable field capture.

The controlled, synthetic evaluation reported here establishes that the geometry is correct and that the resulting quality improvement is substantial and specifically *coupled* between its pose and focal-length components – a finding with a direct design implication for any similar system: apply both corrections jointly, not one in isolation. It does not, and cannot, establish real-world validity on its own; Section 8 lays out the specific field studies – paired ground-truth capture, controlled ablation, blind perceptual evaluation, cross-device benchmarking, and a dedicated parallax-sensitivity protocol – required before that claim can be made.

Looking beyond this paper's scope, the taxonomy of Table 1 and the error decomposition of Section 7.4 point directly at what remains: parallax-tolerant, depth-aware composition (addressing E4, the error this method structurally cannot touch) is the largest open technical direction, and is precisely the kind of problem – an inertial prior available, an on-device efficiency budget imposed, real field data accumulating from a deployed tool – for which no directly applicable prior method yet exists in the literature surveyed in Section 2.

Acknowledgments

[Add acknowledgments here if applicable.]

References

[1] M. Brown and D. G. Lowe, "Automatic Panoramic Image Stitching using Invariant Features," *International Journal of Computer Vision*, vol. 74, no. 1, pp. 59–73, 2007.

[2] R. Szeliski, "Image Alignment and Stitching: A Tutorial," *Foundations and Trends in Computer Graphics and Vision*, vol. 2, no. 1, pp. 1–104, 2006.

[3] Q. Yang, C. Wang, Y. Gao, H. Qu, and E. Y. Chang, "Inertial sensors aided image alignment and stitching for panorama on mobile phones," in *Proc. 1st Int. Workshop on Mobile Location-Based Service (MLBS)*, 2011. DOI: 10.1145/2025876.2025882.

[4] Yaseen, O.-J. Kwon, J. Lee, F. Ullah, S. Jamil, and J. S. Kim, "Automatic Sequential Stitching of High-Resolution Panorama for Android Devices Using Precapture Feature Detection and the Orientation Sensor," *Sensors*, vol. 23, no. 2, art. 879, 2023.

[5] Y. Ding, D. Barath, and Z. Kukelova, "Minimal Solutions for Panoramic Stitching Given Gravity Prior," in *Proc. IEEE/CVF Int. Conf. on Computer Vision (ICCV)*, 2021. arXiv:2012.00465.

[6] L. Nie, C. Lin, K. Liao, S. Liu, and Y. Zhao, "Unsupervised Deep Image Stitching: Reconstructing Stitched Features to Images," *IEEE Transactions on Image Processing*, vol. 30, pp. 6184–6197, 2021.

[7] L. Nie, C. Lin, K. Liao, S. Liu, and Y. Zhao, "Parallax-Tolerant Unsupervised Deep Image Stitching," in *Proc. IEEE/CVF Int. Conf. on Computer Vision (ICCV)*, 2023, pp. 7399–7408. arXiv:2302.08207.

[8] H. Li, K. Luo, B. Zeng, and S. Liu, "GyroFlow+: Gyroscope-Guided Unsupervised Deep Homography and Optical Flow Learning," *International Journal of Computer Vision*, 2023. arXiv:2301.10018.

[9] P. Purkait, T.-J. Chin, and I. Reid, "NeuRoRA: Neural Robust Rotation Averaging," in *Proc. European Conf. on Computer Vision (ECCV)*, 2020. arXiv:1912.04485.

[10] G. Potje, F. Cadar, A. Araujo, R. Martins, and E. R. Nascimento, "XFeat: Accelerated Features for Lightweight Image Matching," in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2024. arXiv:2404.19174.

[11] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, "LightGlue: Local Feature Matching at Light Speed," in *Proc. IEEE/CVF Int. Conf. on Computer Vision (ICCV)*, 2023, pp. 17627–17638. arXiv:2306.13643.

[12] G. Wahba, "A Least Squares Estimate of Satellite Attitude," *SIAM Review*, vol. 7, no. 3, p. 409, 1965.

[13] F. L. Markley, "Attitude Determination Using Vector Observations and the Singular Value Decomposition," *The Journal of the Astronautical Sciences*, vol. 36, no. 3, pp. 245–258, 1988.

[14] G. Casiez, N. Roussel, and D. Vogel, "1 Euro Filter: A Simple Speed-Based Low-Pass Filter for Noisy Input in Interactive Systems," in *Proc. SIGCHI Conf. on Human Factors in Computing Systems (CHI '12)*, Austin, TX, USA, 2012, pp. 2527–2530. DOI: 10.1145/2207676.2208639.

A Notation Summary

Symbol	Meaning
N	Number of source photographs in a capture session ($N \leq 26$)
W, H	Photograph width and height, pixels
f	Focal length, pixels
θ_h	Horizontal field of view
$\mathbf{R}_i \in \mathrm{SO}(3)$	World-frame rotation of view i (refined)
$\tilde{\mathbf{R}}_i \in \mathrm{SO}(3)$	Recorded device-orientation rotation of view i (prior)
$\mathbf{R}_{ij}^{\mathrm{rel}}$	Relative rotation from view i to view j
$\log(\cdot), \exp(\cdot)$	$\mathrm{SO}(3) \leftrightarrow \mathbb{R}^3$ tangent-space maps (Rodrigues' formula)
$w_{\mathrm{tilt}}, w_{\mathrm{yaw}}$	Anisotropic bundle-adjustment prior weights
E	Set of retained correspondence edges (view pairs)
g_i	Estimated exposure gain for view i

Table 7: Notation used throughout Sections 4–5.

B Reproducibility

All three offline experiments of Section 7 are implemented as deterministic, seeded automated test suites within the application's source repository, runnable with Node.js and requiring no browser: a pure-geometry suite (Experiment 1), a real-model fixture suite consuming recorded ONNX inference outputs (Experiment 2), and an end-to-end image-quality decomposition suite that writes its stitched panoramas to disk for visual inspection (Experiment 3). A fourth suite covers the orientation filter of Section 7.5. The refinement pipeline's production code (feature extraction, matching, calibration, and bundle adjustment) is loaded directly by all suites from the same modules the deployed application ships, rather than a parallel reimplementation, specifically so the two cannot silently drift apart.

Two categories of defect described in this paper could not be caught by any of the above, because they involve a real browser rather than pure computation: the WebAssembly runtime actually loading and executing on-device, and the WebGL viewer actually rendering. These are covered by two additional browser-driven regression tests, executed headlessly against the real, unmodified shipping modules – one that loads the refinement worker end to end (real runtime, real model, real inference) and one that renders a synthetic panorama with known ceiling and floor colours through the real viewer, dispatches real pointer events to look up and down, and reads the rendered canvas back. Both defects in Section 7.5 were of a kind that produces confident, plausible-looking, wrong output, which is precisely the case where an assertion against ground truth is worth more than inspection.

The capture pattern is duplicated between the capture module and the offline harnesses (the latter needs it without loading the capture UI). That copy went stale once during this work – it remained at the previous 26-shot pattern after the capture side moved to 34 – causing the evaluation to silently report coverage for a pattern the application no longer used. The test suite now extracts and executes the capture module's real pattern function and asserts the two agree exactly, which is a more reliable guarantee than a comment asking future maintainers to keep them synchronized.